



Bônus — Arquivos DBF nativos em Harbour

Jornada DEV START · Bônus do Módulo 6 · 21/07 · Programa START — TOTVS

Paulista Material extra, não entra na correção automática. Não é pré-requisito para o Módulo 7. É pra você guardar e explorar no seu próprio ritmo.

Por que isso importa (mesmo sem aula dedicada)

Você está prestes a entrar no Protheus, que guarda **todos** os dados em tabelas (SA1, SB1, SZ1...) usando um formato de arquivo chamado **DBF** (*Data Base File*) — o mesmo formato usado pelo clássico **dBASE/Clipper**, avô do Harbour.

O Protheus **abstrai** boa parte disso pra você (via **AxCadastro**, **mBrowse**, dicionário de dados), mas por baixo do capô ele continua abrindo, lendo e gravando **arquivos .dbf** — os mesmos comandos que você vai usar aqui em Harbour puro.



Entender DBF nativo agora ajuda a entender **por que** o dicionário de dados do Protheus (Módulo 7) existe: ele é, essencialmente, uma camada de configuração **em cima** de arquivos DBF — os mesmos conceitos, com um framework a mais.

O arquivo DBF: um array de registros gravado em disco

Lembra dos **arrays multidimensionais** do início desta aula (Módulo 6)? Um arquivo DBF é exatamente essa ideia — só que **persistida em disco**, em vez de sumir quando o programa fecha:

Array em memória:

```
{ {"Ana", 25, .T.},  
  {"Bruno", 30, .F.} }
```

→

Arquivo DBF em disco:

CLIENTES.DBF

NOME	IDADE	ATIVO
Ana	25	.T.
Bruno	30	.F.

Cada **linha** é um **registro**. Cada **coluna** é um **campo**, com um **tipo** definido (Character, Numeric, Date, Logical, Memo) — a mesma ideia dos tipos que você já usa em variáveis.

Criando uma tabela DBF

Usa-se `DBCcreate()`, passando o nome do arquivo e a **estrutura** dos campos (um array de arrays):

```
FUNCTION Main()
  LOCAL aEstrutura := {;
    { "NOME", "C", 30, 0 },,; // Character, 30 posições
    { "IDADE", "N", 3, 0 },,; // Numeric, 3 dígitos, 0 decimais
    { "ATIVO", "L", 1, 0 } } // Logical

  DbCreate( "CLIENTES.DBF", aEstrutura )
  QQout( "Tabela criada!" )
RETURN NIL
```

Cada linha da estrutura é: `{ NomeCampo, Tipo, Tamanho, Decimais }`.

Tipo	Letra	Exemplo
Caractere	C	nome, código
Numérico	N	idade, preço
Data	D	data de nascimento
Lógico	L	ativo/inativo
Memo	M	texto longo

Abrindo, gravando e lendo registros

```
FUNCTION Main()
    // Abre a tabela (cria a área de trabalho)
    dbUseArea( .T., "DBFCDX", "CLIENTES.DBF", "CLIENTES", .T., .F. )

    // Adiciona um novo registro (em branco) e grava os campos
    dbAppend()
    FieldPut( 1, "Ana Souza" )    // campo 1 = NOME
    FieldPut( 2, 25 )             // campo 2 = IDADE
    FieldPut( 3, .T. )           // campo 3 = ATIVO

    // Percorre todos os registros do início ao fim
    dbGoTop()
    DO WHILE !Eof()
        QOut( FieldGet(1), FieldGet(2), FieldGet(3) )
        dbSkip()
    ENDDO

    dbCloseArea()
    RETURN NIL
```

Ou, de forma mais legível, usando o alias da tabela:

```
CLIENTES->NOME    // lê o campo NOME do registro atual
CLIENTES->IDADE := 26 // grava no campo IDADE (precisa estar em modo de edição)
```

Buscando um registro (índice + dbSeek)

Percorrer registro por registro (`dbSkip`) é lento em tabelas grandes. Por isso se usa **índice** — o mesmo conceito do SIX que você vai ver no Protheus:

```

FUNCTION Main()
  dbUseArea( .T., "DBFCDX", "CLIENTES.DBF", "CLIENTES", .T., .F. )

  // Cria um índice pelo campo NOME
  dbCreateIndex( "CLIENTES", "NOME", "NOME", .F. )

  // Busca direta (rápida) em vez de percorrer tudo
  dbSeek( "Ana Souza" )
  IF Found()
    QOut( "Encontrado! Idade: " + Str( CLIENTES->IDADE ) )
  ELSE
    QOut( "Não encontrado." )
  ENDIF

  dbCloseArea()
RETURN NIL

```

De DBF nativo para o Protheus — o que muda

Harbour puro (hoje)	Protheus (a partir do Módulo 7)
<code>DbCreate()</code> + array de estrutura	Configurar campos no dicionário (SX3) — o Protheus cria o DBF por você
<code>dbUseArea()</code>	<code>dbSelectArea()</code> — a tabela já existe, você só abre
<code>dbAppend()</code> + <code>FieldPut()</code> manual	<code>AxInclui</code> / <code>AxCadastro</code> fazem isso pela tela
<code>dbCreateIndex()</code> na hora	Índices já configurados no SIX
Sem multiempresa	Todo registro tem <code>XX_FILIAL</code> (múltiplas empresas no mesmo DBF)

A “mágica” do Protheus é justamente **automatizar** tudo isso que você acabou de fazer na mão.

Resumo rápido

```
// Criar
DbCreate("ARQ.DBF", { {"CAMPO","C",10,0}, ... })

// Abrir / Fechar
dbUseArea(.T., "DBFCDX", "ARQ.DBF", "ALIAS", .T., .F.)
dbCloseArea()

// Navegar
dbGoTop() · dbSkip() · dbGoBottom() · Eof() · Bof()

// Ler / Gravar
FieldGet(nPos) / ALIAS->CAMPO
FieldPut(nPos, valor) / ALIAS->CAMPO := valor
dbAppend()           // novo registro em branco

// Buscar
dbCreateIndex("ALIAS","NOME_IDX","CAMPO",.F.)
dbSeek("valor") · Found()
```



Não é exercício obrigatório — mas se quiser praticar, crie uma tabela **AGENDA.DBF** com os mesmos campos do seu **mini-controle de estoque** (Exercício 11 do Módulo 6) e adapte pra gravar em disco em vez de em array — você vai sentir na prática por que o Protheus poupa tanto trabalho. 🔍